

Aula Prática 9 – Redes de Coexpressão

Nicholas Barroso

2026-07-18

Índice

0.1	Roteiro de Aula Prática: Análise de Redes de Coexpressão com WGCNA	2
0.1.1	Estrutura da Aula (4 horas)	2
0.1.2	Pré-requisitos para os Alunos	3
0.1.3	Instalação dos Pacotes (antes da aula)	3
0.2	Parte 1 – Correlação (45 minutos)	3
0.2.1	1.1. Preparação dos Dados (15 minutos)	3
0.2.2	1.2. Construção da Matriz de Correlação (15 minutos)	5
0.2.3	1.3. Escolha do Poder (Soft-Thresholding) (15 minutos)	7
0.3	Parte 2 – Rede (1 hora)	9
0.3.1	2.1. Construção da Adjacência (15 minutos)	9
0.3.2	2.2. Transformação para Matriz de Dissimilaridade (15 minutos)	11
0.3.3	2.3. Clustering Hierárquico (30 minutos)	12
0.4	Parte 3 – Módulos (1 hora)	13
0.4.1	3.1. Identificação de Módulos (25 minutos)	13
0.4.2	3.2. Fusão de Módulos Similares (20 minutos)	15
0.4.3	3.3. Análise dos Módulos (15 minutos)	17
0.5	Parte 4 – Hub Genes (45 minutos)	18
0.5.1	4.1. Identificação de Genes Centrais (Hub Genes) (20 minutos)	18
0.5.2	4.2. Identificação de Genes Candidatos (15 minutos)	20
0.5.3	4.3. Visualização da Rede (10 minutos)	23
0.6	Entrega e Interpretação Biológica (30 minutos)	25
0.6.1	4.4. Análise Funcional dos Módulos (Bonus)	25
0.6.2	Itens para Entrega	27
0.6.3	Perguntas Guia para Discussão	28
0.6.4	Dicas Finais para os Alunos	29

1	Introdução e Fluxograma Mermaid para Aula Prática de WGCNA	30
1.1	Introdução	30
1.1.1	Contexto Científico	30
1.1.2	Fundamentos Técnicos	30
1.1.3	Objetivos da Aula	31
1.1.4	Pré-requisitos Técnicos	31
1.2	Fluxograma Mermaid	33
1.3	Descrição do Fluxograma	34
1.3.1	1. Preparação e Correlação (45 minutos)	34
1.3.2	2. Construção da Rede (1 hora)	34
1.3.3	3. Identificação de Módulos (1 hora)	34
1.3.4	4. Análise de Hub Genes (45 minutos)	34
1.3.5	5. Entrega e Discussão (30 minutos)	34
1.3.6	Pontos de Destaque no Fluxograma	35
1.3.7	Benefícios do Fluxograma para a Aula	35

Com certeza! Aqui está o roteiro completo e detalhado para uma aula prática de 4 horas sobre análise de redes de coexpressão gênica usando WGCNA (Weighted Gene Co-expression Network Analysis) no R.

0.1 Roteiro de Aula Prática: Análise de Redes de Coexpressão com WGCNA

Carga horária: 4 horas

Ferramentas: R, WGCNA (Weighted Gene Co-expression Network Analysis)

Objetivo: Capacitar os alunos a construir e interpretar redes de coexpressão gênica, identificar módulos de genes funcionalmente relacionados e determinar genes hub (centrais) para gerar hipóteses biológicas.

0.1.1 Estrutura da Aula (4 horas)

- **Parte 1 (45 minutos):** Correlação e preparação dos dados.
- **Parte 2 (1 hora):** Construção da rede de coexpressão.
- **Parte 3 (1 hora):** Identificação e análise de módulos.
- **Parte 4 (45 minutos):** Determinação de hub genes e genes candidatos.
- **Entrega e Discussão (30 minutos):** Interpretação biológica e apresentação dos resultados.

0.1.2 Pré-requisitos para os Alunos

- Conhecimento básico de R e RStudio.
- Dados de expressão gênica (matriz de contagens ou valores normalizados, ex: RNA-seq ou microarray).
- Pacotes R instalados: WGCNA, ggplot2, igraph, reshape2, corrplot, pheatmap.

0.1.3 Instalação dos Pacotes (antes da aula)

```
# Instalar pacotes CRAN
install.packages(c("WGCNA", "ggplot2", "igraph", "reshape2", "corrplot", "pheatmap"))

# WGCNA depende de pacotes do Bioconductor
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")
BiocManager::install(c("impute", "preprocessCore"))

# Para análise funcional (opcional)
BiocManager::install(c("clusterProfiler", "org.Hs.eg.db", "org.Mm.eg.db"))
```

0.2 Parte 1 – Correlação (45 minutos)

0.2.1 1.1. Preparação dos Dados (15 minutos)

0.2.1.1 Roteiro no R/RStudio:

```
# <mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark>

# PARTE 1: CORRELAÇÃO

# <mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark>
```

```

# 1. Carregar os pacotes
library(WGCNA)
library(ggplot2)
library(reshape2)
library(corrplot)
library(pheatmap)

# 2. Configurações iniciais do WGCNA

# Permite o uso de múltiplos threads (opcional)
enableWGCNAThreads(nThreads = 4)

# 3. Importar os dados de expressão

# Exemplo: matriz com genes nas linhas e amostras nas colunas

# Ou usar dados simulados para demonstração
expr_data <- read.table("expressao_normalizada.txt", header = TRUE, row.names = 1)

# 3.1. Opção: Usar dados simulados para teste rápido (se não tiver dados reais)
if (!exists("expr_data")) {
  set.seed(123)
  n_genes <- 1000
  n_samples <- 30
  expr_data <- matrix(rnorm(n_genes * n_samples, mean = 10, sd = 2),
                      nrow = n_genes, ncol = n_samples)
  rownames(expr_data) <- paste0("Gene", 1:n_genes)
  colnames(expr_data) <- paste0("Sample", 1:n_samples)
  cat("Dados simulados criados!\n")
}

# 4. Verificar a estrutura dos dados
cat("Dimensão dos dados:", dim(expr_data), "\n")
cat("Primeiras 5 linhas e colunas:\n")
print(expr_data[1:5, 1:5])

# 5. Verificar e tratar dados faltantes

```

```

if (any(is.na(expr_data))) {
  cat("Encontrados", sum(is.na(expr_data)), "valores NA\n")
  # Imputar pela média de cada gene
  for (i in 1:nrow(expr_data)) {
    expr_data[i, is.na(expr_data[i, ])] <- mean(expr_data[i, ], na.rm = TRUE)
  }
  cat("Valores NA imputados pela média\n")
}

# 6. Remover genes com baixa variância (opcional, mas recomendado)

# Manter apenas os 5000 genes mais variáveis para reduzir ruído
if (nrow(expr_data) > 5000) {
  vars <- apply(expr_data, 1, var)
  expr_data <- expr_data[order(vars, decreasing = TRUE)[1:5000], ]
  cat("Mantidos os 5000 genes mais variáveis\n")
}

# 7. Transpor: WGCNA espera amostras nas linhas e genes nas colunas
datExpr <- as.data.frame(t(expr_data))
cat("Dados transpostos - Amostras:", nrow(datExpr), "Genes:", ncol(datExpr), "\n")

# 8. Verificar se há problemas com os dados
gsg <- goodSamplesGenes(datExpr, verbose = 3)
if (!gsg$allOK) {
  # Remover genes e amostras com problemas
  datExpr <- datExpr[gsg$goodSamples, gsg$goodGenes]
  cat("Removidos genes/amostras com problemas\n")
}

```

0.2.2 1.2. Construção da Matriz de Correlação (15 minutos)

```

# 1. Calcular a matriz de correlação entre todos os genes

# Método de Pearson (padrão) ou Spearman (para dados não lineares)
cat("Calculando matriz de correlação...\n")
cor_matrix <- cor(datExpr, method = "pearson", use = "pairwise.complete.obs")

```

```

cat("Matriz de correlação criada: dimensão", dim(cor_matrix), "\n")

# 2. Visualizar a matriz de correlação (para um subconjunto)

# Selecionar os 50 primeiros genes para visualização
n_genes_plot <- min(50, ncol(cor_matrix))
cor_subset <- cor_matrix[1:n_genes_plot, 1:n_genes_plot]

# 3. Plotar heatmap da correlação
png("correlation_matrix_heatmap.png", width = 1000, height = 1000)
corrplot(cor_subset,
          method = "color",
          type = "upper",
          order = "hclust",
          tl.col = "black",
          tl.cex = 0.7,
          title = paste("Matriz de Correlação - Top", n_genes_plot, "Genes"),
          mar = c(0, 0, 2, 0))
dev.off()
cat("Heatmap da correlação salvo como 'correlation_matrix_heatmap.png'\n")

# 4. Versão alternativa com pheatmap (mais bonito)
annotation_row <- data.frame(
  Gene = rownames(cor_subset),
  row.names = rownames(cor_subset)
)
pheatmap(cor_subset,
          main = paste("Matriz de Correlação - Top", n_genes_plot, "Genes"),
          color = colorRampPalette(c("blue", "white", "red"))(100),
          clustering_method = "average",
          filename = "correlation_heatmap_pheatmap.png",
          width = 10, height = 10)

# 5. Análise rápida: distribuição das correlações
cor_vector <- cor_matrix[upper.tri(cor_matrix)]
cat("\nEstatísticas das correlações:\n")
cat("Mínimo:", min(cor_vector, na.rm = TRUE), "\n")
cat("Máximo:", max(cor_vector, na.rm = TRUE), "\n")

```

```

cat("Média:", mean(cor_vector, na.rm = TRUE), "\n")
cat("Mediana:", median(cor_vector, na.rm = TRUE), "\n")
cat("Desvio padrão:", sd(cor_vector, na.rm = TRUE), "\n")

# 6. Histograma das correlações
png("correlation_distribution.png", width = 800, height = 600)
hist(cor_vector,
     breaks = 50,
     col = "lightblue",
     main = "Distribuição das Correlações entre Genes",
     xlab = "Correlação de Pearson",
     ylab = "Frequência")
abline(v = 0, col = "red", lty = 2, lwd = 2)
abline(v = mean(cor_vector, na.rm = TRUE), col = "blue", lty = 2, lwd = 2)
legend("topright",
     legend = c("Zero", paste("Média =", round(mean(cor_vector, na.rm = TRUE), 3))),
     col = c("red", "blue"),
     lty = 2,
     lwd = 2)
dev.off()

```

0.2.3 1.3. Escolha do Poder (Soft-Thresholding) (15 minutos)

```

# 1. Escolher o poder de adjacência (soft-thresholding power)

# WGCNA transforma a correlação em adjacência usando uma função de potência:

# Adjacência = |cor|^{#lst:power}

# 2. Testar diferentes poderes para encontrar o que torna a rede "scale-free"
powers <- c(seq(1, 10, by = 1), seq(12, 20, by = 2))
cat("Testando poderes:", paste(powers, collapse = ", "), "\n")

# 3. Calcular a métrica de scale-free para cada poder
cat("Calculando métricas de scale-free...\n")
sft <- pickSoftThreshold(datExpr,
                        powerVector = powers,

```

```

        verbose = 5,
        networkType = "signed")

# 4. Visualizar os resultados

# Gráfico 1:  $R^2$  da scale-free topology vs poder
png("soft_threshold_analysis.png", width = 1000, height = 800)
par(mfrow = c(1, 2))

# Gráfico 1:  $R^2$ 
plot(sft$fitIndices[, 1],
     -sign(sft$fitIndices[, 3]) * sft$fitIndices[, 2],
     xlab = "Soft Threshold (power)",
     ylab = "Scale Free Topology Model Fit,  $R^2$ ",
     main = "Escolha do Poder para Rede Scale-Free",
     type = "n")
text(sft$fitIndices[, 1],
     -sign(sft$fitIndices[, 3]) * sft$fitIndices[, 2],
     labels = powers,
     col = "red",
     cex = 1.2)

# Adicionar linha de corte (geralmente  $R^2 > 0.8$ )
abline(h = 0.8, col = "blue", lty = 2, lwd = 2)
abline(h = 0.9, col = "green", lty = 2, lwd = 2)

# Gráfico 2: Conectividade média
plot(sft$fitIndices[, 1],
     sft$fitIndices[, 5],
     xlab = "Soft Threshold (power)",
     ylab = "Mean Connectivity",
     type = "n",
     main = "Conectividade Média vs Poder")
text(sft$fitIndices[, 1],
     sft$fitIndices[, 5],
     labels = powers,
     col = "red",
     cex = 1.2)

```



```

dev.off()

# 5. Escolher o poder: primeiro valor onde  $R^2 > 0.8$ 
picked_power <- sft$powerEstimate

# Se não encontrou, usar um valor padrão
if (is.na(picked_power) || picked_power < 1) {
  # Escolher o primeiro poder onde  $R^2 > 0.8$ 
  r2_values <- -sign(sft$fitIndices[, 3]) * sft$fitIndices[, 2]
  idx <- which(r2_values > 0.8)[1]
  if (!is.na(idx)) {
    picked_power <- sft$fitIndices[idx, 1]
  } else {
    # Fallback: usar poder = 6 (comum para dados de expressão)
    picked_power <- 6
    cat("Usando poder padrão = 6\n")
  }
}

cat("\n<mark>= Poder selecionado:", picked_power, "</mark>=\n")

```

0.3 Parte 2 – Rede (1 hora)

0.3.1 2.1. Construção da Adjacência (15 minutos)

```

# <mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark>

# PARTE 2: REDE DE COEXPRESSION

# <mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark>

# 1. Construir a matriz de adjacência usando o poder selecionado

# A adjacência é uma medida de força da conexão entre genes

```

```

cat("Construindo matriz de adjacência...\n")
adjacency <- adjacency(datExpr,
                      power = picked_power,
                      type = "signed") # ou "unsigned"

cat("Matriz de adjacência criada: dimensão", dim(adjacency), "\n")

# 2. Verificar a esparsidade da rede
density <- sum(adjacency > 0) / (ncol(adjacency)^2)
cat("Densidade da rede:", density * 100, "%\n")
cat("Número de conexões não-zero:", sum(adjacency > 0), "\n")

# 3. Verificar a distribuição da adjacência
adj_values <- adjacency[upper.tri(adjacency)]
cat("\nEstatísticas da adjacência:\n")
cat("Mínimo:", min(adj_values), "\n")
cat("Máximo:", max(adj_values), "\n")
cat("Média:", mean(adj_values), "\n")
cat("Mediana:", median(adj_values), "\n")

# 4. Visualizar um subconjunto da matriz de adjacência
n_genes_plot <- min(30, ncol(adjacency))
png("adjacency_matrix_heatmap.png", width = 1000, height = 1000)
heatmap.2(adjacency[1:n_genes_plot, 1:n_genes_plot],
          main = paste("Matriz de Adjacência (Top", n_genes_plot, "Genes)"),
          trace = "none",
          col = colorRampPalette(c("white", "yellow", "red"))(100),
          margins = c(10, 10))
dev.off()
cat("Heatmap da adjacência salvo como 'adjacency_matrix_heatmap.png'\n")

# 5. Análise da topologia da rede

# Calcular graus (degree) de cada nó
degrees <- colSums(adjacency > 0)
cat("\nEstatísticas dos graus (degree):\n")
cat("Mínimo:", min(degrees), "\n")
cat("Máximo:", max(degrees), "\n")

```

```

cat("Média:", mean(degrees), "\n")
cat("Mediana:", median(degrees), "\n")

# 6. Plotar distribuição dos graus (para verificar scale-free)
png("degree_distribution.png", width = 800, height = 600)
hist(degrees,
      breaks = 30,
      col = "lightgreen",
      main = "Distribuição dos Graus dos Genes",
      xlab = "Degree (número de conexões)",
      ylab = "Frequência")
dev.off()

```

0.3.2 2.2. Transformação para Matriz de Dissimilaridade (15 minutos)

```

# 1. WGCNA usa TOM (Topological Overlap Matrix) para medir similaridade

# TOM considera não apenas conexões diretas, mas também compartilhamento de vizinhos
cat("Calculando TOM (Topological Overlap Matrix)...\n")
TOM <- TOMsimilarity(adjacency, TOMType = "signed")
cat("TOM calculado: dimensão", dim(TOM), "\n")

# 2. Converter para dissimilaridade (1 - similaridade)
dissTOM <- 1 - TOM

# 3. Visualizar a dissimilaridade TOM

# (importante para ver a estrutura da rede)
n_genes_plot <- min(100, ncol(dissTOM))
png("dissTOM_heatmap.png", width = 1000, height = 1000)
heatmap.2(dissTOM[1:n_genes_plot, 1:n_genes_plot],
          main = paste("Dissimilaridade TOM (Top", n_genes_plot, "Genes)"),
          trace = "none",
          col = colorRampPalette(c("white", "black"))(100),
          margins = c(10, 10))
dev.off()
cat("Heatmap da dissimilaridade TOM salvo como 'dissTOM_heatmap.png'\n")

```

```
# 4. Salvar a matriz TOM para uso posterior
save(TOM, file = "TOM_matrix.RData")
cat("Matriz TOM salva em 'TOM_matrix.RData'\n")
```

0.3.3 2.3. Clustering Hierárquico (30 minutos)

```
# 1. Realizar clustering hierárquico baseado na dissimilaridade TOM
cat("Realizando clustering hierárquico...\n")
geneTree <- hclust(as.dist(dissTOM), method = "average")
cat("Clustering concluído!\n")

# 2. Plotar o dendrograma completo
png("dendrograma_completo.png", width = 1200, height = 800)
plot(geneTree,
      xlab = "",
      sub = "",
      main = "Dendrograma de Clustering - Todos os Genes",
      labels = FALSE,
      hang = 0.04,
      cex.main = 1.5)
dev.off()
cat("Dendrograma completo salvo como 'dendrograma_completo.png'\n")

# 3. Salvar o dendrograma em PDF para melhor qualidade
pdf("dendrograma_completo.pdf", width = 14, height = 8)
plot(geneTree,
      xlab = "",
      sub = "",
      main = "Dendrograma de Clustering - Todos os Genes",
      labels = FALSE,
      hang = 0.04)
dev.off()

# 4. Análise da estrutura do dendrograma
```

```

# Número de clusters naturais (corte em diferentes alturas)
heights <- seq(0.1, 1, by = 0.1)
n_clusters <- sapply(heights, function(h) {
  length(unique(cutree(geneTree, h = h)))
})

# Plotar número de clusters vs altura de corte
png("clusters_vs_height.png", width = 800, height = 600)
plot(heights, n_clusters,
      type = "b",
      pch = 19,
      col = "blue",
      xlab = "Altura de Corte",
      ylab = "Número de Clusters",
      main = "Número de Clusters vs Altura de Corte")
grid()
dev.off()

cat("\nNúmero de clusters em diferentes alturas:\n")
for (i in 1:length(heights)) {
  cat("Altura", heights[i], ":", n_clusters[i], "clusters\n")
}

```

0.4 Parte 3 – Módulos (1 hora)

0.4.1 3.1. Identificação de Módulos (25 minutos)

```

# <mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark>

# PARTE 3: MÓDULOS DE COEXPRESSÃO

# <mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark>

# 1. Identificar módulos usando corte dinâmico de árvore

```

```

# O método Dynamic Tree Cut identifica clusters automaticamente
cat("Identificando módulos com Dynamic Tree Cut...\n")

dynamicMods <- cutreeDynamic(dendro = geneTree,
                             distM = dissTOM,
                             deepSplit = 2,      # Sensibilidade (1-4)
                             pamRespectsDendro = FALSE,
                             minClusterSize = 30) # Tamanho mínimo do módulo

# 2. Ver o número de módulos identificados
table_mods <- table(dynamicMods)
cat("\n<mark>= Módulos identificados </mark>=\n")
cat("Número total de módulos:", length(table_mods) - 1, "\n")
cat("Módulo 0 (não atribuídos):", table_mods["0"], "genes\n")
for (i in 1:length(table_mods)) {
  mod_name <- names(table_mods)[i]
  if (mod_name != "0") {
    cat("Módulo", mod_name, ":", table_mods[i], "genes\n")
  }
}

# 3. Converter para cores (cada módulo recebe uma cor)
dynamicColors <- labels2colors(dynamicMods)
table(dynamicColors)

# 4. Plotar o dendrograma com as cores dos módulos
pdf("dendrograma_modulos_dinamicos.pdf", width = 14, height = 8)
plotDendroAndColors(geneTree,
                    dynamicColors,
                    "Módulos Dinâmicos",
                    dendroLabels = FALSE,
                    hang = 0.03,
                    addGuide = TRUE,
                    guideHang = 0.05,
                    main = "Dendrograma com Módulos Identificados")
dev.off()
cat("Dendrograma com módulos salvo como 'dendrograma_modulos_dinamicos.pdf'\n")

```

```

# 5. Salvar versão em PNG
png("dendrograma_modulos_dinamicos.png", width = 1200, height = 800)
plotDendroAndColors(geneTree,
                    dynamicColors,
                    "Módulos Dinâmicos",
                    dendroLabels = FALSE,
                    hang = 0.03,
                    addGuide = TRUE,
                    guideHang = 0.05)

dev.off()

# 6. Análise rápida dos módulos
module_colors_initial <- unique(dynamicColors)
module_colors_initial <- module_colors_initial[module_colors_initial != "grey"]
cat("\nCores dos módulos iniciais:", paste(module_colors_initial, collapse = ", "), "\n")

```

0.4.2 3.2. Fusão de Módulos Similares (20 minutos)

```

# 1. Calcular a correlação entre os módulos (usando eigengenes)
cat("Calculando eigengenes dos módulos...\n")
MEList <- moduleEigengenes(datExpr,
                          colors = dynamicColors,
                          softPower = picked_power,
                          nPC = 1)

MEs <- MEList$eigengenes

# 2. Calcular a correlação entre eigengenes
MEDiss <- 1 - cor(MEs)
METree <- hclust(as.dist(MEDiss), method = "average")

# 3. Plotar o dendrograma dos módulos
pdf("dendrograma_modulos_eigengenes.pdf", width = 10, height = 6)
plot(METree,
     main = "Dendrograma de Módulos Baseado em Eigengenes",
     xlab = "",
     sub = "",
     cex.main = 1.5)

```

```

abline(h = 0.25, col = "red", lty = 2, lwd = 2)
dev.off()
cat("Dendrograma dos eigengenes salvo como 'dendrograma_modulos_eigengenes.pdf'\n")

# 4. Definir um limiar para fusão (geralmente 0.25 ou 0.5)

# Módulos com dissimilaridade < 0.25 (correlação > 0.75) serão mesclados
mergeCutHeight <- 0.25
cat("Altura de corte para fusão:", mergeCutHeight, "\n")

# 5. Mesclar módulos similares
cat("Mesclando módulos similares...\n")
merge <- mergeCloseModules(datExpr,
                           dynamicColors,
                           cutHeight = mergeCutHeight,
                           verbose = 3)

# 6. Obter as cores dos módulos mesclados
mergedColors <- merge$colors
mergedMEs <- merge$newMEs

# 7. Verificar quantos módulos foram mesclados
cat("\n<mark>= Módulos após mesclagem </mark>=\n")
table_mods_merged <- table(mergedColors)
cat("Número total de módulos:", length(table_mods_merged) - 1, "\n")
cat("Módulo grey (não atribuídos):", table_mods_merged["grey"], "genes\n")
for (color in names(table_mods_merged)) {
  if (color != "grey") {
    cat("Módulo", color, ":", table_mods_merged[color], "genes\n")
  }
}

# 8. Comparar módulos antes e depois da mesclagem
pdf("dendrograma_modulos_comparacao.pdf", width = 14, height = 8)
plotDendroAndColors(geneTree,
                   cbind(dynamicColors, mergedColors),
                   c("Módulos Dinâmicos", "Módulos Mesclados"),

```



```

        dendroLabels = FALSE,
        hang = 0.03,
        addGuide = TRUE,
        guideHang = 0.05)

dev.off()
cat("Comparação de módulos salva como 'dendrograma_modulos_comparacao.pdf'\n")

# 9. Salvar os módulos finais
module_colors <- unique(mergedColors)
module_colors <- module_colors[module_colors != "grey"]
cat("\nCores finais dos módulos:", paste(module_colors, collapse = ", "), "\n")

```

0.4.3 3.3. Análise dos Módulos (15 minutos)

```

# 1. Salvar os genes de cada módulo
module_genes <- list()
for (color in module_colors) {
  genes_in_module <- colnames(datExpr)[mergedColors == color]
  module_genes[color](color) <- genes_in_module
  cat("Módulo", color, ":", length(genes_in_module), "genes\n")
}

# 2. Salvar listas de genes em arquivos
for (color in names(module_genes)) {
  write.table(module_genes[color](color),
    file = paste0("module_genes_", color, ".txt"),
    row.names = FALSE,
    col.names = FALSE,
    quote = FALSE)
}
cat("Listas de genes dos módulos salvas em arquivos .txt\n")

# 3. Calcular eigengenes finais (representam a expressão do módulo)
MEs_final <- moduleEigengenes(datExpr, mergedColors)$eigengenes

# 4. Correlacionar módulos entre si

```

```

ME_cor <- cor(MEs_final)
ME_cor_pheatmap <- pheatmap(ME_cor,
                             main = "Correlação entre Módulos",
                             color = colorRampPalette(c("blue", "white", "red"))(100),
                             clustering_method = "average",
                             display_numbers = TRUE,
                             number_format = "%.2f",
                             filename = "module_correlations.png",
                             width = 10, height = 10)
cat("Mapa de correlação entre módulos salvo como 'module_correlations.png'\n")

# 5. Se tiver dados de fenótipos/traits, correlacionar módulos com eles

# Exemplo (com dados simulados):
if (FALSE) { # Desativado por padrão
  # Criar dados de exemplo
  trait_data <- data.frame(
    Age = rnorm(nrow(datExpr), mean = 50, sd = 10),
    Sex = factor(sample(c("M", "F"), nrow(datExpr), replace = TRUE)),
    Condition = factor(sample(c("Control", "Treatment"), nrow(datExpr), replace = TRUE))
  )

  # Correlacionar módulos com traits
  trait_cor <- cor(MEs_final, trait_data$Age, use = "p")
  print(trait_cor)
}

```

0.5 Parte 4 – Hub Genes (45 minutos)

0.5.1 4.1. Identificação de Genes Centrais (Hub Genes) (20 minutos)

```

# <mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark>

# PARTE 4: HUB GENES E GENES CANDIDATOS

# <mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark>

```

```

# 1. Calcular a conectividade intramodular (kWithin)

# kWithin = soma das adjacências com todos os genes do mesmo módulo
cat("Calculando conectividades intramodulares...\n")
kWithin <- intramodularConnectivity(adjacency, mergedColors)
colnames(kWithin) <- c("kWithin", "kOut", "kDiff")

# 2. Calcular a conectividade total (kTotal)
kTotal <- apply(adjacency, 2, sum)
names(kTotal) <- colnames(adjacency)

# 3. Calcular a proximidade aos eigengenes (module membership)

# MM = correlação entre a expressão do gene e o eigengene do módulo
cat("Calculando Module Membership...\n")
MM <- cor(datExpr, MEs_final, use = "p")
colnames(MM) <- paste0("MM.", colnames(MEs_final))

# 4. Identificar hub genes de cada módulo

# Hub genes = alta conectividade intramodular + alta MM
cat("\n<mark>= IDENTIFICANDO HUB GENES </mark>=\n")
hub_genes <- list()

for (color in module_colors) {
  # Genes do módulo
  genes_in_mod <- colnames(datExpr)[mergedColors == color]

  # Conectividade e MM desses genes
  k_mod <- kWithin[genes_in_mod, "kWithin"]
  MM_mod <- MM[genes_in_mod, paste0("MM.", color)]

  # Combinar em um dataframe
  hub_df <- data.frame(
    Gene = genes_in_mod,
    kWithin = k_mod,
    MM = MM_mod,
    kTotal = kTotal[genes_in_mod]
  )
}

```

```

)

# Ordenar por conectividade (top hub genes)
hub_df <- hub_df[order(hub_df$kWithin, decreasing = TRUE), ]

# Selecionar top 10
hub_genes[color](color) <- head(hub_df, 10)

cat("\n<mark>= Módulo", color, "- Top 10 Hub Genes </mark>=\n")
print(hub_genes[color](color))
}

# 5. Salvar lista de hub genes
for (color in names(hub_genes)) {
  write.csv(hub_genes[color](color),
            file = paste0("hub_genes_", color, ".csv"),
            row.names = FALSE)
}
cat("Listas de hub genes salvas em arquivos CSV\n")

# 6. Visualizar hub genes em um heatmap

# Criar uma matriz com os top genes de cada módulo
top_genes_all <- data.frame()
for (color in names(hub_genes)) {
  temp <- hub_genes[color](color)[1:3, ] # Top 3 de cada módulo
  temp$Module <- color
  top_genes_all <- rbind(top_genes_all, temp)
}

# Salvar top genes combinados
write.csv(top_genes_all, "top_hub_genes_all_modules.csv", row.names = FALSE)

```

0.5.2 4.2. Identificação de Genes Candidatos (15 minutos)

```

# 1. Selecionar módulos de interesse

```

```

# Critérios: módulos grandes, com alta conectividade, etc.
cat("\n<mark>= ANALISANDO MÓDULOS PARA SELEÇÃO </mark>=\n")

module_stats <- data.frame(
  Module = module_colors,
  Size = sapply(module_genes, length),
  Avg_kWithin = sapply(module_colors, function(c) {
    mean(kWithin[module_genes[c](c), "kWithin"])
  }),
  Avg_MM = sapply(module_colors, function(c) {
    mean(MM[module_genes[c](c), paste0("MM.", c)])
  })
)

# Ordenar por tamanho
module_stats <- module_stats[order(module_stats$Size, decreasing = TRUE), ]
print(module_stats)

# Selecionar o maior módulo como alvo
target_module <- module_stats$Module[1]
cat("\nMódulo alvo selecionado:", target_module,
    "(tamanho:", module_stats[module_stats$Module == target_module, "Size"], "genes)\n")

# 2. Extrair genes candidatos do módulo alvo
candidate_genes <- hub_genes[target_module](target_module)

# 3. Critérios adicionais para genes candidatos

# a) kWithin > mediana + 2*SD (genes altamente conectados)
kWithin_median <- median(kWithin[, "kWithin"])
kWithin_sd <- sd(kWithin[, "kWithin"])
threshold_kWithin <- kWithin_median + 2 * kWithin_sd

# b) MM > 0.7 (forte associação com o módulo)
threshold_MM <- 0.7

```

```

# c) kTotal > mediana (genes globalmente importantes)
kTotal_median <- median(kTotal)

# Aplicar filtros
candidate_genes$is_central <- candidate_genes$kWithin > threshold_kWithin
candidate_genes$is_strong_MM <- candidate_genes$MM > threshold_MM
candidate_genes$is_high_total <- candidate_genes$kTotal > kTotal_median

# Genes que atendem a todos os critérios
final_candidates <- candidate_genes[
  candidate_genes$is_central &
  candidate_genes$is_strong_MM &
  candidate_genes$is_high_total,
]

cat("\n<mark>= GENES CANDIDATOS FINAIS </mark>=\n")
cat("Critérios:\n")
cat(" - kWithin >", round(threshold_kWithin, 3), "\n")
cat(" - MM >", threshold_MM, "\n")
cat(" - kTotal >", round(kTotal_median, 3), "\n")
cat("\nNúmero de genes candidatos:", nrow(final_candidates), "\n")

if (nrow(final_candidates) > 0) {
  print(final_candidates)
} else {
  cat("Nenhum gene atendeu a todos os critérios.\n")
  cat("Selecionando os top 5 genes por kWithin como candidatos.\n")
  final_candidates <- head(candidate_genes, 5)
  print(final_candidates)
}

# 4. Exportar genes candidatos
write.csv(final_candidates,
  file = paste0("candidate_genes_", target_module, ".csv"),
  row.names = FALSE)
cat("Genes candidatos salvos em 'candidate_genes_', target_module, '.csv'\n")

# 5. Salvar todos os genes do módulo alvo para análise funcional

```

```
writeLines(module_genes[target_module](target_module),
           paste0("module_genes_target_", target_module, ".txt"))
```

0.5.3 4.3. Visualização da Rede (10 minutos)

```
# 1. Selecionar módulo para visualização
selected_module <- target_module
genes_selected <- module_genes[selected_module](selected_module)
cat("\nVisualizando módulo", selected_module,
    "com", length(genes_selected), "genes\n")

# 2. Extrair sub-rede
adj_sub <- adjacency(genes_selected, genes_selected)

# 3. Usar o igraph para visualização
library(igraph)

# Criar grafo
graph <- graph_from_adjacency_matrix(adj_sub,
                                     mode = "undirected",
                                     weighted = TRUE,
                                     diag = FALSE)

# Simplificar (remover múltiplas arestas)
graph <- simplify(graph)

# 4. Calcular métricas da rede
cat("\nMétricas da rede do módulo", selected_module, ":\n")
cat("Número de vértices:", vcount(graph), "\n")
cat("Número de arestas:", ecount(graph), "\n")
cat("Densidade:", edge_density(graph), "\n")
cat("Transitividade (clusterização):", transitivity(graph), "\n")

# 5. Definir atributos dos vértices
```

```

# Tamanho baseado na conectividade
V(graph)$size <- log(degree(graph) + 1) * 3

# Cor baseada no módulo
V(graph)$color <- selected_module

# Destacar hub genes
hub_gene_names <- hub_genes[selected_module](selected_module)$Gene[1:5]
V(graph)$hub <- ifelse(V(graph)$name %in% hub_gene_names, "Hub", "Other")

# 6. Definir layout
set.seed(123)
layout_graph <- layout_with_fr(graph)

# 7. Plotar a rede completa
pdf(paste0("network_module_", selected_module, ".pdf"), width = 12, height = 12)
plot(graph,
      vertex.label = NA,
      vertex.size = V(graph)$size,
      vertex.color = ifelse(V(graph)$hub == "Hub", "red", selected_module),
      vertex.frame.color = ifelse(V(graph)$hub == "Hub", "darkred", "gray"),
      vertex.frame.width = ifelse(V(graph)$hub == "Hub", 3, 1),
      edge.width = E(graph)$weight * 5,
      edge.color = rgb(0.5, 0.5, 0.5, 0.3),
      main = paste("Rede de Coexpressão - Módulo", selected_module),
      layout = layout_graph)
dev.off()

# 8. Versão com labels dos hub genes
pdf(paste0("network_module_", selected_module, "_hub_labels.pdf"), width = 12, height = 12)
plot(graph,
      vertex.label = ifelse(V(graph)$hub == "Hub", V(graph)$name, NA),
      vertex.label.cex = 0.8,
      vertex.label.color = "darkred",
      vertex.label.dist = 2,
      vertex.size = V(graph)$size,
      vertex.color = ifelse(V(graph)$hub == "Hub", "red", selected_module),

```



```

vertex.frame.color = ifelse(V(graph)$hub == "Hub", "darkred", "gray"),
vertex.frame.width = ifelse(V(graph)$hub == "Hub", 3, 1),
edge.width = E(graph)$weight * 3,
edge.color = rgb(0.5, 0.5, 0.5, 0.2),
main = paste("Hub Genes em Destaque - Módulo", selected_module),
layout = layout_graph)
dev.off()

cat("Figuras da rede salvas em PDF\n")

# 9. Salvar versões em PNG
png(paste0("network_module_", selected_module, ".png"), width = 1200, height = 1200)
plot(graph,
      vertex.label = NA,
      vertex.size = V(graph)$size,
      vertex.color = ifelse(V(graph)$hub == "Hub", "red", selected_module),
      vertex.frame.color = ifelse(V(graph)$hub == "Hub", "darkred", "gray"),
      vertex.frame.width = ifelse(V(graph)$hub == "Hub", 3, 1),
      edge.width = E(graph)$weight * 5,
      edge.color = rgb(0.5, 0.5, 0.5, 0.3),
      main = paste("Rede de Coexpressão - Módulo", selected_module),
      layout = layout_graph)
dev.off()

```

0.6 Entrega e Interpretação Biológica (30 minutos)

0.6.1 4.4. Análise Funcional dos Módulos (Bonus)

```

# <mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark>

# ANÁLISE FUNCIONAL (BONUS)

# <mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark>

# 1. Análise de enriquecimento funcional com clusterProfiler

```

```

# Nota: Isso requer genes em formato Entrez ID

# Verificar se os genes estão em formato de símbolo ou Entrez

# Se estiverem em símbolo, converter
if (FALSE) { # Desativado por padrão - requer pacotes adicionais
  library(clusterProfiler)
  library(org.Hs.eg.db) # Para humanos

  # Exemplo: analisar o módulo alvo
  genes_for_enrich <- module_genes[target_module](target_module)

  # Converter para Entrez IDs
  gene_ids <- bitr(genes_for_enrich,
    fromType = "SYMBOL",
    toType = "ENTREZID",
    OrgDb = org.Hs.eg.db)

  # Enriquecimento GO (Biological Process)
  go_enrich <- enrichGO(gene = gene_ids$ENTREZID,
    OrgDb = org.Hs.eg.db,
    ont = "BP",
    pAdjustMethod = "BH",
    pvalueCutoff = 0.05,
    qvalueCutoff = 0.10,
    readable = TRUE)

  # Visualizar resultados
  if (!is.null(go_enrich) && nrow(go_enrich) > 0) {
    # Dotplot
    pdf("enrichment_GO_dotplot.pdf", width = 10, height = 8)
    dotplot(go_enrich, showCategory = 20, title = "GO Enrichment - Biological Process")
    dev.off()

    # Barplot
    pdf("enrichment_GO_barplot.pdf", width = 10, height = 8)
    barplot(go_enrich, showCategory = 20, title = "GO Enrichment - Biological Process")
    dev.off()

    # Salvar resultados
    write.csv(as.data.frame(go_enrich),

```

```
file = paste0("enrichment_GO_", target_module, ".csv"),
row.names = FALSE)

cat("Análise de enriquecimento GO concluída!\n")
} else {
cat("Nenhum enriquecimento GO significativo encontrado.\n")
}
}
```

0.6.2 Itens para Entrega

```
# <mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark>

# PREPARAÇÃO DOS ARQUIVOS PARA ENTREGA

# <mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark><mark></mark>

# 1. Criar um resumo da análise
sink("analise_wgcna_resumo.txt")

cat("<mark>= ANÁLISE WGCNA - RESUMO </mark>=\n\n")
cat("Data da análise:", date(), "\n\n")

cat("<mark>= PARÂMETROS </mark>=\n")
cat("Poder (soft threshold):", picked_power, "\n")
cat("Altura de corte para fusão:", mergeCutHeight, "\n")
cat("Tamanho mínimo do módulo:", 30, "\n")
cat("Número total de genes:", ncol(datExpr), "\n")
cat("Número de amostras:", nrow(datExpr), "\n\n")

cat("<mark>= MÓDULOS </mark>=\n")
cat("Número total de módulos:", length(module_colors), "\n")
for (color in module_colors) {
    cat("Módulo", color, ":", length(module_genes[color](color)), "genes\n")
}
cat("\n")

cat("<mark>= HUB GENES </mark>=\n")
for (color in names(hub_genes)) {
```

```

    cat("\nMódulo", color, "- Top 3 Hub Genes:\n")
    print(hub_genes[color](color)[1:3, c("Gene", "kWithin", "MM")])
}
cat("\n")

cat("<mark>= GENES CANDIDATOS </mark>=\n")
cat("Módulo alvo:", target_module, "\n")
if (nrow(final_candidates) > 0) {
    print(final_candidates[, c("Gene", "kWithin", "MM", "kTotal")])
} else {
    cat("Nenhum gene candidato identificado.\n")
}

sink()

cat("Resumo da análise salvo em 'analise_wgcna_resumo.txt'\n")

# 2. Lista de arquivos gerados
cat("\n<mark>= ARQUIVOS GERADOS </mark>=\n")
cat("1. correlation_matrix_heatmap.png - Heatmap da correlação\n")
cat("2. correlation_distribution.png - Distribuição das correlações\n")
cat("3. soft_threshold_analysis.png - Análise do poder\n")
cat("4. adjacency_matrix_heatmap.png - Heatmap da adjacência\n")
cat("5. degree_distribution.png - Distribuição dos graus\n")
cat("6. dissTOM_heatmap.png - Heatmap da dissimilaridade TOM\n")
cat("7. dendrograma_completo.png/pdf - Dendrograma completo\n")
cat("8. dendrograma_modulos_dinamicos.png/pdf - Dendrograma com módulos\n")
cat("9. dendrograma_modulos_eigengenes.pdf - Dendrograma dos eigengenes\n")
cat("10. dendrograma_modulos_comparacao.pdf - Comparação de módulos\n")
cat("11. module_correlations.png - Correlação entre módulos\n")
cat("12. module_genes_*.txt - Listas de genes por módulo\n")
cat("13. hub_genes_*.csv - Tabelas de hub genes\n")
cat("14. candidate_genes_*.csv - Genes candidatos\n")
cat("15. network_module_*.pdf/png - Figuras da rede\n")
cat("16. analise_wgcna_resumo.txt - Resumo da análise\n")

```

0.6.3 Perguntas Guia para Discussão

1. Análise dos Módulos:

- Quantos módulos foram identificados?

- Qual o tamanho médio dos módulos?
- Existem módulos muito grandes ou muito pequenos?

2. Interpretação Biológica:

- Os hub genes identificados são conhecidos na literatura?
- Quais vias biológicas estão enriquecidas nos módulos?
- Há módulos específicos que podem estar associados a fenótipos de interesse?

3. Validação dos Resultados:

- Como validar experimentalmente os genes candidatos?
- Quais experimentos poderiam ser realizados para confirmar os achados?

4. Aplicações:

- Como essa análise pode gerar novas hipóteses?
- Quais genes poderiam ser alvos terapêuticos ou biomarcadores?

0.6.4 Dicas Finais para os Alunos

1. **Documentação:** Consulte o manual do WGCNA: `vignette("WGCNA")`

2. **Parâmetros:** Experimente diferentes valores de:

- `softPower`: afeta a sensibilidade da rede
- `deepSplit`: controla a granularidade dos módulos
- `minClusterSize`: tamanho mínimo dos módulos
- `mergeCutHeight`: nível de fusão de módulos similares

3. **Interpretação:** A análise de rede é uma ferramenta exploratória. Os resultados devem ser validados experimentalmente.

4. **Visualização:** Use ferramentas como:

- Cytoscape para visualizações mais sofisticadas
- Gephi para redes interativas
- ggplot2 para gráficos personalizados

5. **Publicação:** Para publicar, inclua:

- Parâmetros usados (`power`, `minClusterSize`, `mergeCutHeight`)
- Estatísticas da rede (número de módulos, tamanhos)
- Lista de hub genes e suas funções
- Análise de enriquecimento funcional

Este roteiro fornece uma base completa para a análise de redes de coexpressão com WGCNA, desde a preparação dos dados até a interpretação biológica, capacitando os alunos a realizar análises robustas e gerar hipóteses científicas.

1 Introdução e Fluxograma Mermaid para Aula Prática de WGCNA

1.1 Introdução

A análise de redes de coexpressão gênica utilizando WGCNA (Weighted Gene Co-expression Network Analysis) é uma abordagem poderosa em bioinformática e biologia de sistemas que permite identificar padrões de coexpressão entre genes em larga escala. Esta técnica é fundamental para:

1.1.1 Contexto Científico

- **Descoberta de módulos funcionais:** Agrupa genes com padrões de expressão similares, que frequentemente participam das mesmas vias biológicas ou processos celulares
- **Identificação de genes hub:** Genes centrais que podem atuar como reguladores chave ou biomarcadores
- **Integração com fenótipos:** Correlaciona módulos genéticos com características clínicas ou condições experimentais
- **Geração de hipóteses:** Fornece insights sobre mecanismos moleculares subjacentes a fenótipos de interesse

1.1.2 Fundamentos Técnicos

O WGCNA se baseia em três princípios principais:

1. **Topologia Scale-Free:** Redes biológicas geralmente seguem uma distribuição de lei de potência, onde poucos genes têm muitas conexões (hubs) e muitos genes têm poucas conexões
2. **Soft-Thresholding:** Ao invés de usar um corte binário para definir conexões (presente/ausente), o WGCNA utiliza uma função de potência para preservar a continuidade das correlações
3. **Topological Overlap Measure (TOM):** Captura não apenas conexões diretas entre genes, mas também a similaridade de seus padrões de vizinhança, proporcionando uma medida mais robusta de coexpressão

1.1.3 Objetivos da Aula

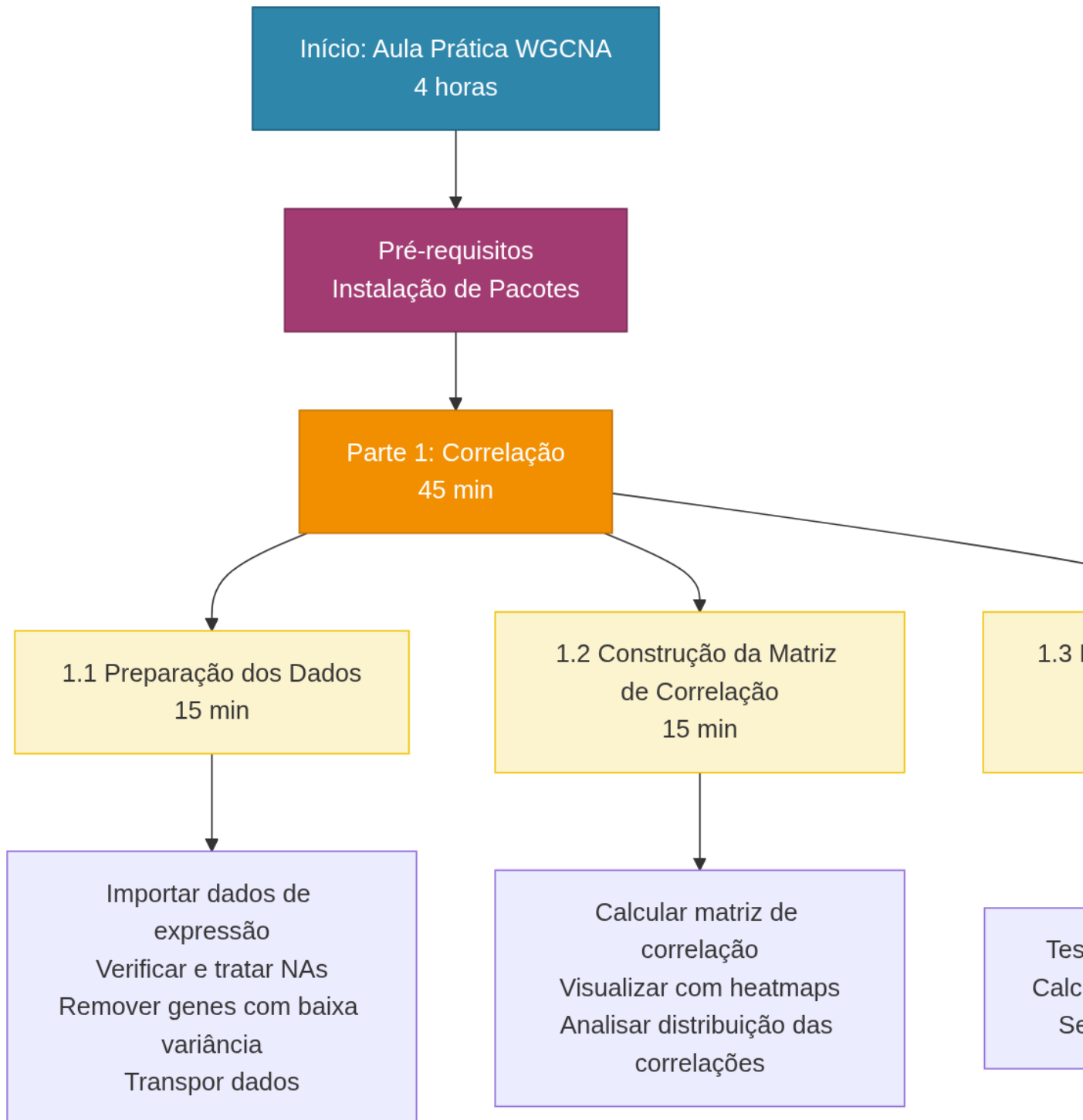
Esta aula prática de 4 horas tem como objetivos:

- **Familiarizar** os alunos com o pacote WGCNA no R
- **Capacitar** os alunos a construir e interpretar redes de coexpressão
- **Desenvolver** habilidades para identificar módulos funcionais e genes hub
- **Aplicar** conceitos estatísticos e computacionais em dados reais de expressão gênica
- **Gerar** hipóteses biológicas testáveis a partir dos resultados

1.1.4 Pré-requisitos Técnicos

- Conhecimento básico de R e RStudio
 - Pacotes R instalados: WGCNA, ggplot2, igraph, reshape2, corrplot, pheatmap
 - Dados de expressão gênica (matriz de contagens ou valores normalizados)
-

1.2 Fluxograma Mermaid



1.3 Descrição do Fluxograma

O fluxograma acima representa a estrutura completa da aula prática de 4 horas, organizada em 5 etapas principais:

1.3.1 1. Preparação e Correlação (45 minutos)

- Preparação e limpeza dos dados de expressão gênica
- Construção da matriz de correlação entre genes
- Escolha do poder ótimo (soft-thresholding) para garantir topologia scale-free

1.3.2 2. Construção da Rede (1 hora)

- Transformação da correlação em adjacência usando o poder selecionado
- Cálculo da matriz TOM (Topological Overlap Measure)
- Clustering hierárquico e análise do dendrograma

1.3.3 3. Identificação de Módulos (1 hora)

- Aplicação do Dynamic Tree Cut para identificar módulos
- Fusão de módulos similares baseada em correlação de eigengenes
- Análise descritiva dos módulos identificados

1.3.4 4. Análise de Hub Genes (45 minutos)

- Cálculo de conectividades intramodulares
- Identificação dos top 10 hub genes por módulo
- Seleção de genes candidatos baseada em múltiplos critérios
- Visualização da rede do módulo alvo

1.3.5 5. Entrega e Discussão (30 minutos)

- Análise funcional (enriquecimento GO) como atividade bônus
- Preparação dos arquivos para entrega
- Discussão dos resultados e interpretação biológica

1.3.6 Pontos de Destaque no Fluxograma

- **Cores distintas** para cada parte da aula facilitam a navegação
- **Tempos indicados** em cada etapa auxiliam no gerenciamento da aula
- **Sub-etapas detalhadas** mostram as atividades específicas
- **Setas de fluxo** indicam a sequência lógica da análise
- **Círculos de decisão** mostram onde parâmetros podem ser ajustados
- **Links para documentação** e ferramentas complementares no final

1.3.7 Benefícios do Fluxograma para a Aula

1. **Visão geral clara:** Os alunos podem visualizar todo o processo antes de começar
2. **Gerenciamento de tempo:** Cada etapa tem duração definida
3. **Progressão lógica:** As etapas seguem uma sequência que faz sentido estatístico
4. **Flexibilidade:** Permite ajustes baseados no progresso da turma
5. **Referência rápida:** Durante a aula, os alunos podem consultar o fluxograma para saber em que etapa estão

Este fluxograma serve como um guia visual abrangente que complementa o roteiro detalhado, ajudando tanto os instrutores quanto os alunos a navegar pela complexa análise de redes de coexpressão com WGCNA.